

LayerLeap — Supercharging LayerSkip

Final Project Report

Parth Arora¹
partharora²

Garrett Manaster¹
gmanaster³²

Andrew Peng¹
apeng43²

Prabhav Gupta¹
pgupta359²

Richard So¹
so²

¹Georgia Institute of Technology
²@gatech.edu

Abstract

Large Language Models (LLMs) have become prevalent, but their sequential token generation during inference leads to high computational cost and latency, limiting their practicality. LayerSkip addresses this by introducing self-speculative decoding, where an initial draft of tokens from early layers is verified by deeper layers in parallel, significantly improving inference speed. This paper investigates three approaches to further optimize it and improve token throughput: Dynamic Layer Skipping, which uses Bayesian optimization to identify optimal configurations of skipped and exit layers; Layer Importance, where we quantify the contribution of individual layers to prioritize skipping less significant ones; and Hierarchical Speculative Decoding, a novel extension of the self-speculative strategy. Our preliminary results across various natural language processing tasks demonstrate the potential of these techniques to further accelerate LLM inference, contributing to more efficient and deployable AI systems.

1. Introduction

Large Language Models (LLMs) like ChatGPT [1] and LLaMA [4] have gained widespread popularity in recent years. However, their inference processes are computationally expensive and suffer from high latency.

This is primarily because LLMs process each token sequentially through every layer of the model during inference. This design limits token throughput—the number of tokens generated per second—and results in slow response times. LayerSkip [2] proposes an efficient fine-tuning strategy that addresses this by introducing early exit loss and layer dropout.

A key innovation of LayerSkip is its *self-speculative decoding* strategy. Instead of running each token through the entire model, it first drafts a prediction using the first E lay-

ers and then verifies it in parallel using the remaining layers. This method achieves more than 2× faster inference [2] while preserving output quality. By doing so, LayerSkip improves not just speed but also model interpretability and efficiency, making LLMs more practical for real-world use.

However, we believe there is still room for improvement. Our research explores three approaches to further optimize inference:

1. **Dynamic Layer Skipping** – We apply Bayesian optimization [14] to determine the optimal combination of layers to skip, exit layer, and number of speculations to maximize token throughput.
2. **Layer Importance** – We quantify the relative importance of each layer and prioritize skipping the least important ones.
3. **Hierarchical Speculative Decoding** – We propose a new speculative strategy that builds on top of the self-speculation strategy introduced in LayerSkip [2] to further optimize the process.

1.1. Importance

Speeding up inference for LLMs is critical because these models are increasingly integrated into industry and everyday applications. Their high computational cost, however, limits accessibility, scalability, and real-time usage.

Improving upon LayerSkip, especially for widely-used models like LLaMA, can reduce energy consumption, improve environmental sustainability, and unlock new applications previously deemed too resource-intensive. Ultimately, this work has the potential to accelerate AI deployment across industries, democratize access to advanced language models, and contribute to more efficient, scalable, and impactful AI systems.

2. Related Works

LayerSkip introduces multiple strategies to accelerate generation for decoder-only language models [2]. To fur-

ther optimize this process, our work builds on existing approaches and identifies opportunities for deeper enhancements.

2.1. Dynamic Layer Skipping

A core idea in LayerSkip is its mechanism for early exits and layer skipping. During training, LayerSkip applies a progressively increasing layer dropout rate, pushing earlier layers to learn stronger token generation capabilities. It also introduces auxiliary loss terms at intermediate layers by projecting the output of the Transformer layer with the LM head and applying Cross Entropy with the expected tokens. This allows earlier exits during inference without major accuracy degradation.

However, LayerSkip assumes the ability to fine-tune the model, which may not always be feasible. In contrast, some methods provide plug-and-play acceleration. The Draft & Verify framework [16] uses Bayesian Optimization (BO) [14] to identify layer subsets that optimize speed while retaining high accuracy. Their approach drafts tokens using a subset of layers (not necessarily contiguous) and verifies them with the whole model, while waiving the need for the model to be retrained or fine tuned with this generation strategy. In short, BO is a robust technique for optimizing black-box, expensive-to-evaluate functions, such as the throughput of the model with different layer skipping configurations [16]. We integrate their BO-based layer selection strategy into LayerSkip to dynamically find optimal layers to skip prior to the exit layer parameter with hopes of further increasing token throughput.

2.2. Layer Importance

Layer pruning offers another effective method for reducing model size and latency. Early pruning techniques date back to LeCun et al. [7], who introduced Optimal Brain Damage. More recent works like LayerDrop [3] regularize training by randomly dropping layers, which improves robustness and enables efficient inference-time pruning. This builds on earlier ideas of Dropout [13] and structured pruning in deep CNNs [6]. Several modern approaches quantify layer importance before pruning. For instance, ShortGPT [10] computes a Block Influence (BI) score using hidden states from a calibration set and prunes layers with the lowest influence. They achieve a 25% reduction in computation while retaining 90% of performance. Similarly, LLM Pruner [9] identifies minimal removable groups in models like LLaMA-7B, assigns importance scores, and applies LoRA [5] for post-pruning fine-tuning—retaining 95% performance after 20% parameter removal. We plan to adopt the BI score from ShortGPT to identify and prune layers with minimal contribution, improving token throughput while preserving quality.

2.3. Hierarchical Speculative Decoding

Speculative Decoding is another line of work aimed at reducing inference time. It involves a fast draft model that proposes γ completions, which are then verified by the full model in a single forward pass [8]. This significantly reduces autoregressive latency. Self-Speculative Decoding, as introduced in LayerSkip and Draft & Verify, implements this idea but using the same model for both drafting and verifying, early exiting for the draft stage and using the full layers for verification [2]. BO is used to tune which layers to skip based on empirical latency measurements. We want to build on this and improve this with Hierarchical Speculative Decoding strategy discussed in next section.

3. Method / Approach

3.1. Dynamic Layer Skipping

Our approach was to combine LayerSkip’s early exiting and Draft & Verify’s skipped layers during the draft stage. During the draft stage, LayerSkip only uses the first n layers to draft tokens, before verifying all of them at once during the verify stage. We combine that approach with Draft & Verify’s method of skipping certain layers in the draft stage. As a result, during the draft stage, we only use a subset of the first n layers. The verify stage stays the same; all of the model’s layers are used to verify, which ensures that the accuracy does not drop. A sample diagram of our pipeline is shown in Figure 2.

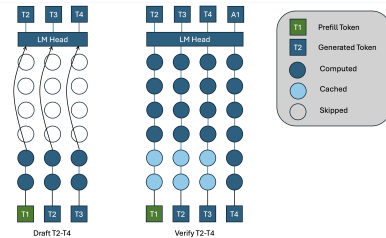


Figure 1: Original LayerSkip Pipeline with `exit_layer = 2`, `num_speculations = 3`

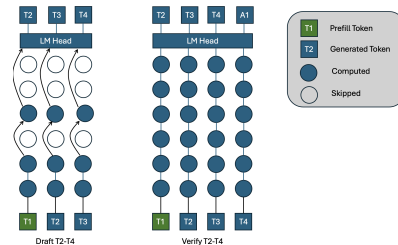


Figure 2: DLS Implementation with `exit_layer = 4`, `num_speculations = 3`, `skipped_layers = [3]`

One of the strengths of LayerSkip was its reuse of the KV cache. Because it was always computing the first n layers during the draft stage, it was able to reuse the KV cache from those first n layers during the verify stage, as seen in Figure 1. With our new implementation, because we cannot assume that all of the first n layers were computed, we had to eliminate the KV cache reuse and force the model to recompute all of the layers during the verify stage. We recognize the potential loss in throughput as a result and evaluated the impact in our experiments.

Similar to Draft & Verify, we use BO to determine which of the first n layers to keep, and which to skip. Recall that BO is an optimization technique for obscure, expensive functions. We define this function with its inputs being an array of n floats $(x_1, x_2, \dots, x_n)$, and its outputs being the resulting throughput of the whole model with verification. For all $0 \leq i \leq n$, if $x_i < 0.5$, the model will skip the i^{th} layer during inference. Note that the `exit_layer` and `num_speculations` parameters remain as constants throughout the optimization process. With the function defined, BO proceeds to balance between exploring new inputs to better approximate the surface of the function, and exploiting known inputs that further improves the objective value. Ideally, through this method we are able to implicitly skip some less "helpful" layers within the first n layers, which will theoretically increase throughput.

3.2. Layer Importance

We aimed to improve inference efficiency by selectively skipping transformer layers based on their importance. Our hypothesis was that layers which perform minimal transformations could be skipped without significantly affecting model performance. To quantify a layer's importance, we examined the similarity between its input and output hidden states. We explored methods from ShortGPT [10], LayerDrop [3], and LLM Pruner [9].

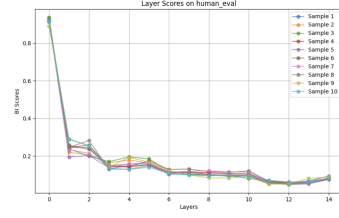
Ultimately, we adopted the *Block Influence (BI)* Score introduced in ShortGPT. This metric quantifies the transformation performed by each layer using cosine similarity between its input and output hidden states. The BI score for the i -th layer is defined as:

$$BI_i = 1 - \mathbb{E}_{X,t} \left[\frac{X_{i,t}^T X_{i+1,t}}{\|X_{i,t}\|_2 \|X_{i+1,t}\|_2} \right], \quad (1)$$

where $X_{i,t}$ is the t -th hidden state vector at layer i . A lower BI score indicates high cosine similarity, suggesting the layer contributes minimally to the model's representation transformation and can potentially be skipped. Based on the paper, empirical studies show a strong correlation between BI score and layer importance.

We started with experimenting using the LLaMA 3.2B model on H100 GPUs, processing 10 samples from each

dataset to observe BI score variation across layers (see Figure 3).



(a) HumanEval Coding [15]

Figure 3: BI Scores across different tasks

We experimented with skipping layers below a certain threshold and observed an increase in throughput for certain cases. As a result, we decided to use this as a metric to measure layer importance. We then conducted more detailed experiments on skipping layers, using the BI score as a heuristic to improve throughput.

3.3. Hierarchical Speculative Decoding

Our first approach is to optimize self speculation on the LayerSkip checkpoints by recursively applying draft and verify. In the original LayerSkip paper [2], they train the LLM such that the hidden states from earlier layers can be unembedded and optimized for the next token prediction task. Their draft model is the first E_1 layers and their verification model is the entire model, leveraging computation re-use by using the KV cache and queries from their draft model in their verification stage. Our approach is to explore how this technique can be applied recursively: the draft model formed from the first E_1 layers can be treated as the verification model, for a new draft model formed from the first $E_2 < E_1$ layers. The KV cache and exit query cache can be strategically re-used to minimize the recomputation from drafting at different layers.

Consider a model with exit layers $E_1 = 14$ and $E_2 = 7$. We draft `num_speculations` tokens from the first E_2 layers, then verify with the first E_1 layers. We repeat this process `num_calls` times, yielding us at most $(\text{num_speculations} + 1) * \text{num_calls}$ verified tokens at the E_1 layer. Then, we treat these as draft tokens and verify them using the entire model. An example of the process is shown in 4 and 5.

For hierarchical speculative decoding, the key parameters include `exit_layers` (exit layer i is treated as the draft model for exit layer $i + 1$), `num_speculations` (the number of speculations to compute for the base draft model with exit layer $i = 0$), and `num_calls` (how many times to sequentially retrieve draft tokens from the submodel before verifying).

Our recursive function computes the verified tokens at exit layer i , given verified tokens up until exit layer $i - 1$. It

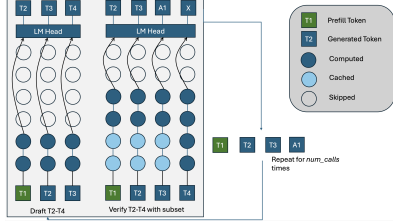


Figure 4: Hierarchical self speculation exit layers 2, 4, 6. $num_speculations=3$, $num_calls=2$, when drafting

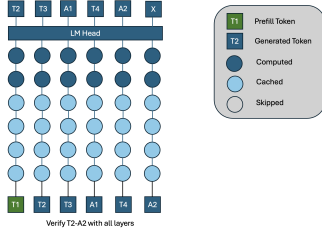


Figure 5: Hierarchical self speculation exit layers 2, 4, 6. $num_speculations=3$, $num_calls=2$, when verifying

also returns the appropriate KV cache and exit query cache for the verified tokens at exit layer i .

The base case is where we autoregressively draft $num_speculations$ tokens from the first $exit_layers[0]$ layers. Then we forward all these tokens at once using the first $exit_layers[1]$ layers, re-using the KV cache and exit queries cached during the draft stage. Verification is performed by comparing the draft tokens with the verification tokens, accepting tokens until the first mismatch is found. At the mismatch, the verification model provides the correct token so we can also accept that token.

The recursive step at i is similar. Our verification model exits at $exit_layers[i]$, and we get our draft tokens by recursively calling this function on exit layer $i - 1$, num_calls amount of times. We can then perform verification on the draft tokens, returning the verified tokens, updated KV cache, and exit queries.

4. Data

We evaluated our approach using diverse natural language processing benchmarks in both summarization and code generation tasks. These datasets were chosen to align with those used in the original LayerSkip paper [2], allowing us for a direct comparison and consistent evaluation for inference time speedups. For summarization, we used CNN/DailyMail (CNN/DM) [11] and XSUM [12], both of which are standard datasets in evaluating abstractive summarization quality. CNN/DM consists of news articles paired with multi sentence summaries, while XSUM focuses on highly compressed, single sentence summaries.

These datasets test the ability of a model to understand and condense content, and as mentioned, were also the primary evaluation datasets in the LayerSkip paper, making them ideal for benchmarking our modifications to the original framework. Additionally, for code generation, we used HumanEval [15], a dataset composed of Python programming problems, each with a function signature, docstring, and a canonical solution. This dataset evaluates functional correctness via unit tests, providing a robust metric for assessing whether inference-time optimizations preserve the logical integrity of generated outputs. LayerSkip also used HumanEval to measure how their early exiting affected structured generation tasks, which we build upon in our experiments. All datasets are publicly available, widely used in the community, and require no special access or licensing. We did not apply any additional fine-tuning or dataset-specific modifications, ensuring that all performance changes are due to inference-time optimizations alone.

5. Experiments and Results

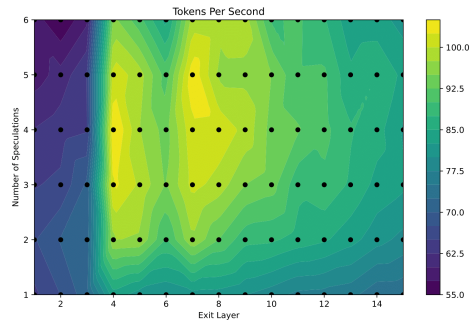
5.1. Dynamic Layer Skipping

5.1.1 Ideal LayerSkip Configurations

In order to evaluate our approach against LayerSkip, we needed to find the combination of hyperparameters ($exit_layer$ and $num_speculations$) that maximized throughput (tokens per second) across all 3 datasets. We ran all of our experiments on the facebook/layer-skip-llama2-7B model. So, we ran a sweep search with these two parameters, running LayerSkip for 50 samples for each dataset. The results for HumanEval is shown in Figure 6, with the rest of the datasets in Figure 16. Clearly, ($exit_layer = 7$, $num_speculations = 5$) is the best set of hyperparameters across all 3 datasets for LayerSkip. This aligns with the findings found by the authors of LayerSkip: for the LayerSkip Llama2-7B model, throughput also peaks at $exit_layer = 7$ (see Figure 15).

5.1.2 Ideal DLS Configurations

In order to find the best combination of hyperparameters ($exit_layer$, $num_speculations$, and $skipped_layers$), we ran Bayesian Optimization across a wide range of $exit_layer$ and $num_speculations$ values (the $skipped_layers$ and throughput would be outputted at the end of the Bayesian Optimization). Similar to the Draft & Verify paper, we ran BO for 1000 iterations with 4 samples from each dataset (12 total) and 32 max generated tokens. Our results can be seen in Figure 7. We can see that plain DLS (Layerskip without KV cache reuse) only performs marginally worse



(a) HumanEval Coding [15]

Figure 6: LayerSkip throughputs for HumanEval Coding.

when compared to LayerSkip. Additionally, DLS w/ BO (skips layers) consistently performs better than LayerSkip’s approach, albeit on a small sample size. Another interesting trend is the DLS w/ BO has a similar performance once the exit layer is high enough. This reduces the need to tune hyperparameters, and we will see later how much DLS w/ BO can outperform LayerSkip with traditionally suboptimal configurations.

We discovered that DLS w/ BO performs best with `exit_layer = 7` and `num_speculations = 6`, skipping only layer 5. We found other similar performing configurations at later exit layers skipping layer 5 as well, along with a contiguous portion of layers leading up to the exit layer (e.g. `exit_layer = 16` skips layers [5, 7, 8, ..., 14, 15]). We postulate that DLS w/ BO prefers to skip layers such that the model only runs on 6-7 layers.

To ensure that the small max tokens value of 32 was not skewing our results, we also ran a sweep with 512 max tokens (same as LayerSkip). The results can be seen in Figure 17 (in the Appendix). The primary difference is that the throughput is larger across all generation strategies (DLS + BO, DLS, LayerSkip, Autoregressive) and hyperparameter configurations. However, the margin of difference between different generation strategies and between different hyperparameter values is roughly the same as with 32 max tokens. So, we conclude that the smaller max tokens does not impact our findings significantly.

5.1.3 Evaluation

Recall that our ideal DLS configuration derived from BO is optimized purely from the 4 data samples of each dataset (CNN/DM, XSUM, and HumanEval). From our sweeps, it appears that DLS w/ BO outperforms all other generation strategies. However, to ensure a more realistic test of DLS w/ BO against LayerSkip and vanilla autoregressive generation, we evaluated all generation methods for 100

randomly-sampled examples from each dataset, for 5 trials. We test LayerSkip’s optimal `exit_layer` configuration (7) against DLS w/ BO with the same configuration with hopes that BO has discovered additional intermediate layers to increase throughput. All trials are run on Georgia Tech’s PACE ICE clusters, with a H200 GPU with 8 CPU cores and 64GB of RAM allocated for execution.

HumanEval	
Generation Strategy	Avg. Throughput
Autoregressive	75.07
LayerSkip (7)	137.11
DLS + BO (7)	142.81

CNN/DM	
Generation Strategy	Avg. Throughput
Autoregressive	74.01
LayerSkip (7)	165.04
DLS + BO (7)	149.36

XSUM	
Generation Strategy	Avg. Throughput
Autoregressive	74.20
LayerSkip (7)	144.96
DLS + BO (7)	142.19

Table 1: Comparison of throughput (tokens/second) averaged over 5 runs over 100 randomly-sampled examples across three evaluation datasets.

From table 1, we see that DLS w/ BO falters against LayerSkip at its most optimal configuration in a more realistic evaluation (with the exception of the HumanEval dataset). Although we tried our best to ensure the same testing environment across each trial, it is possible that external processes made to the clusters could affect our allocated compute. The more convincing argument for this discrepancy is the bias of only optimizing for 4 samples per dataset, following Draft & Verify’s procedure [16].

Despite this, DLS w/ BO holds a significant advantage in adapting to suboptimal `exit_layer` parameters. Typically, tuning this hyperparameter is extremely time-consuming as the practitioner must perform an exhaustive grid search. However, with a large `exit_layer`, BO helps select the optimal layers to use to increase throughput. This is seen in Table 2, where we evaluate LayerSkip and DLS w/ BO at `exit_layer = 16`, a suboptimal hyperparameter according to Figures 15, 6, and 16. Although we do not beat LayerSkip, we trade off a small fraction of performance in exchange for robustness and convenience.

5.2. Layer Importance

5.2.1 Effect of Layer Skipping on Throughput

We conducted some experiments to evaluate the effect of skipping low-BI-score layers to improve throughput (to-

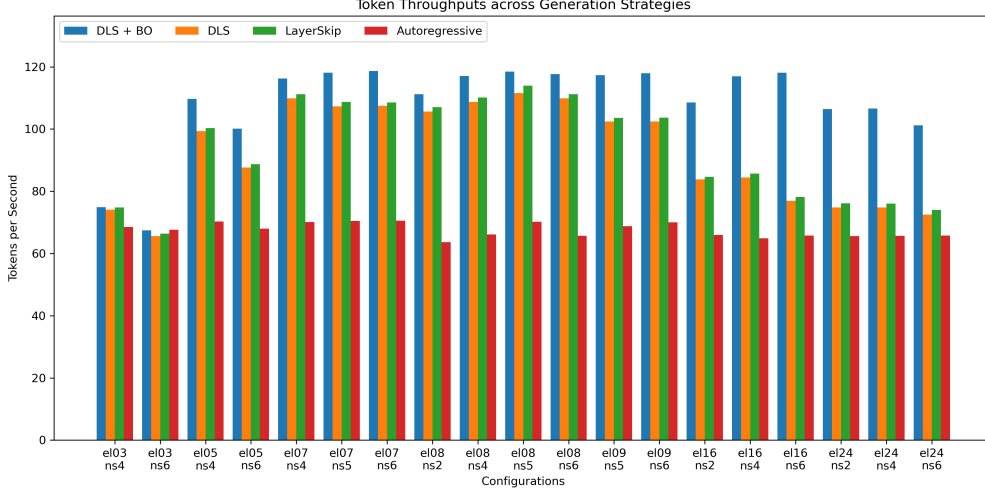


Figure 7: Sweep of different parameters across different generation strategies, including DLS w/ BO after 1000 optimization iterations.

Strategy	Exit Layer	
	7	16
LayerSkip	149.04	109.52
DLS + BO	144.79	145.50

Table 2: Throughput across the 3 datasets between LayerSkip and DLS w/ BO on various exit_layer parameters.

Exit Layer	Num Specs	Layers Skipped	Throughput	
			Before LS	After LS
6	3	3	178.72	190.40
6	5	3	187.53	185.99
7	6	3	147.06	178.79
8	4	3	151.14	178.44
8	3	3, 6	163.16	179.51

Table 3: Throughput before and after layer skipping (LS).

kens/sec). We chose the best-performing layer-skip configurations from our initial sweep over different numbers of speculations and exit layers (see Figure 12). Using our previously mentioned DLS implementation, we skipped layers with the lowest BI scores.

We tried several approaches. Initially, we computed BI scores during the first speculation and skipped layers below a threshold in subsequent iterations. However, this led to BI scores collapsing over iterations eventually all layers were marked to be skipped, with no throughput gains.

Then, we tried selecting the n layers with the lowest BI scores in a benchmark run and re-ran the same benchmark with those layers skipped. We evaluated over 50 combinations and observed several configurations where throughput improved. Table 3 shows a few notable ones:

Interestingly, the best throughput was achieved when skipping only a single layer. This suggests that selectively skipping even a single low-influence layer can offer tangible performance gains without hurting the model.

5.2.2 Scaling to Larger Models

Building on insights from smaller models, we extended our experiments to the LLaMA 2 7B model on H200 GPUs. We swept all combinations of even-numbered speculations and

exit layers. For each configuration, we:

1. Computed BI scores.
2. Identified the single layer (based on previous experiments mentioned) with the lowest BI score.
3. Measured throughput with and without skipping that layer on 10 samples.

We used a combination of the HumanEval [15], CNN/DM Language Summarization [11], and XSUM Summarization [12] datasets, consistent with the LayerSkip [2] paper. We then averaged their results for BI score and throughput measurement. Figure 8 shows the layer with the minimum BI score for each configuration.

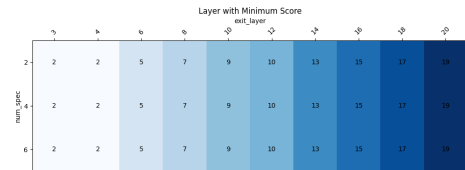


Figure 8: Layer with min score

We then evaluated throughput under both scenarios (with and without layer skipping) and analyzed the relative gain, finding consistent trends similar to our smaller model experiments. Figures 9 and 10 show the result of our sweep measuring throughput without and with skipping the worst layer.

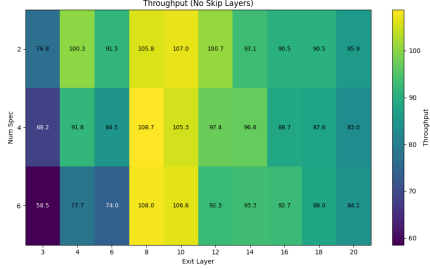


Figure 9: Throughput without skipping layers

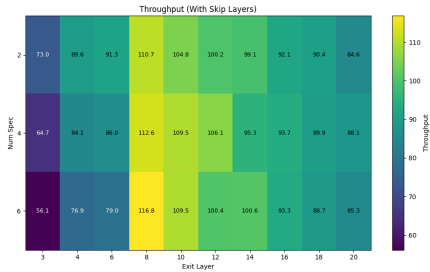


Figure 10: Throughput with skipping layers

Generally there is a slight improvement in throughput especially with higher number of speculations and exit layers. Figure 11 shows throughput comparison with number of speculations being 6 and exit layer more than 6.

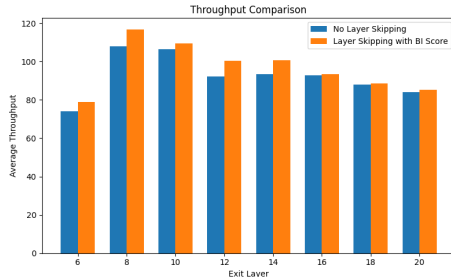


Figure 11: Throughput with skipping layers

There is an average of 4.64% improvement for all cases by just skipping a single layer with lowest BI score.

5.3. Hierarchical Speculative Decoding

5.3.1 Hyperparameter Search

In order to sweep the optimal hyperparameters for hierarchical speculative decoding (which includes a list of exit layers, *num_speculations*, and *num_calls*), we initially tried a grid search, trying all 2 – 4-length subsets of exit layers and a pre-defined list of *num_speculations* and *num_calls*. However, we quickly found that the most optimal hyperparameter with *i*-length exit layers included the most optimal *i* – 1-length exit layers. Thus, by first sweeping with 2 exit layers and then sweeping with 3 exit layers using the best result from the first sweep, we can drastically reduce the search space. Additionally, we pruned any configuration that resulted in an acceptance rate lower than 0.3.

We did a hyperparameter sweep on a smaller subset of each downstream task. Below 12 is a plot comparing tokens per second with different values of exit layer and number of speculations, on CNN/DM summarization [11]:

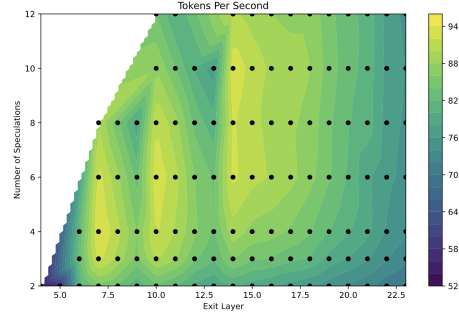


Figure 12: Throughput across different number of speculations and exit layers

Then for hierarchical self speculation, we held the exit layer fixed at $E_1 = 14$ and then did a hyperparameter sweep on exit layer E_2 , number of speculations, and number of calls. Below is a plot with *num_calls* = 3:

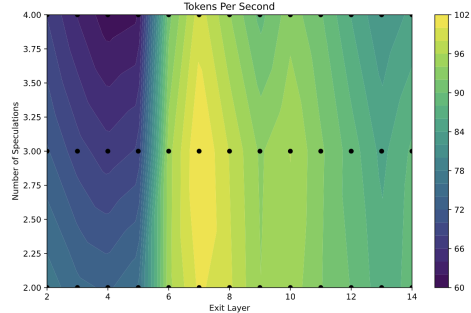


Figure 13: Throughput across different number of speculations and exit layers with $E_1 = 14$ fixed

Generation	Parameters	Tokens/sec	AR	Speedup
CNN/DM summarization				
Autoregressive	N/A	55.3	-	-
Self speculation	$E = 14, d = 10$	91.2	0.71	1.65
Hierarchical self speculation	$E = [7, 14, 40], d = 3, c = 3$	97.5	0.62, 0.78	1.76
CNN/DM LM				
Autoregressive	N/A	60.4	-	-
Self speculation	$E = 7, d = 3$	72.8	0.35	1.21
Hierarchical self speculation	$E = [2, 7, 40], d = 2, c = 2$	62.2	0.26, 0.33	1.03
XSUM				
Autoregressive	N/A	57.7	-	-
Self speculation	$E = 7, d = 3$	88.6	0.55	1.54
Hierarchical self speculation	$E = [6, 7, 40], d = 2, c = 2$	77.5	0.70, 0.47	1.34
HumanEval				
Autoregressive	N/A	59.6	-	-
Self speculation	$E = 7, d = 3$	89.7	0.52	1.51
Hierarchical self speculation	$E = [2, 7, 40], d = 2, c = 2$	74.8	0.23, 0.52	1.26

Table 4: Generation Results with Hierarchical Self Speculation, LLaMA-13B

(a) E is exit layer, d is number of speculations, c is number of calls, AR is acceptance rate

5.3.2 Evaluation

For hierarchical speculative decoding, we evaluated tokens per second on LayerSkip’s LLaMA 2 13B checkpoint on CNN/DM summarization [11], CNN/DM language modeling, XSUM summarization [12], and HumanEval [15]. We used 8 CPUs and 1 H200 for evaluation. We used 64 samples, 128 max number of generated tokens, and no sampling. For self speculation we used exit layer of 14 and 10 speculations, while for hierarchical self speculation we used exit layers of $[7, 14, 40]$, 3 speculations, and 3 sub-model calls. The results are summarized below 4a.

Overall, hierarchical self speculation performed at least as well as autoregressive, and performed better than self speculation only on the CNN/DM summarization task. In general, it doesn’t perform better than self speculation likely due to low acceptance rate which leads to high overhead in computing draft tokens that never get verified. Hierarchical self speculation adds an extra exit layer in between the original exit layer from self speculation and the first layer. Because the first few layers are change the activations of the sequence so much, the acceptance rate is relatively low even for a low number of speculations. Additionally, there are fewer layers to skip between the draft and verify stage (for example for $E = [2, 7, 40]$, the first stage skips 5 layers while the second stage skips 33 layers), leading to a smaller possible speedup over self speculation. We can see this in the results: the only dataset it performed better than self speculation was on CNN/DM summarization, where it was able to have 7 layers in between the first stage’s draft and verify exit layers and had a high acceptance rate of 0.62

and number of speculations of 3.

Thus, increasing the acceptance rate for the lower layers would be instrumental to improving the performance of hierarchical speculative decoding. This may require a novel fine-tuning method. We tried to fine-tune the LayerSkip model by adding a KL divergence term between the last layer and intermediate exit layers on task-specific datasets, but the intermediate results weren’t good and due to a lack of compute power (16 GPU-hour limit on one job) we weren’t able to explore this avenue further.

6. Conclusion

This work builds upon the LayerSkip framework and explores three methods to further optimize LLM inference speed. Dynamic Layer Skipping, using Bayesian Optimization, allows for the intelligent selection of layers to skip during the draft stage, improving throughput. The Layer Importance method, leveraging the Block Influence score, identifies and skips less important layers, also contributing to faster inference. Our experiments indicate that even skipping a single low-influence layer can lead to tangible performance gains. Finally, we introduced Hierarchical Speculative Decoding as a novel approach to build on self-speculative decoding. Our results show the potential of these techniques to significantly improve the efficiency of LLMs during inference time, making them more suitable for real-world applications and contributing to the democratization of advanced AI. Future work will focus on further refining these methods and evaluating their performance on larger models and more complex tasks.

Student Name	Contributed Aspects	Details
Parth Arora	Layer Importance	Conducted initial literature review, implemented code from multiple papers to measure layer importance, developed code to implement a block influence metric and applied it to layer skipping, and performed further experimentation and analysis to gather results.
Garrett Manaster	Block influence sweep and analysis	Implemented grid search sweep to collect baseline block influence, and then to select the layer with the minimum block influence score for a range of number of speculations and exit layer hyperparameters. Ran script on PACE and collected results.
Andrew Peng	Hierarchical self speculation	Extended the LayerSkip repository by implementing the hierarchical self speculation inference technique, verified its correctness, and ran benchmarks across several datasets comparing it to autoregressive and self speculation.
Prabhav Gupta	Dynamic Layer Skipping	Worked with Richard So to modify the LayerSkip pipeline to skip layers during the draft stage. Ran numerous Bayesian Optimization sweeps on PACE across the relevant hyperparameters and created visuals to compare the results to the other generation strategies.
Richard So	DLS & Bayesian Optimization implementation and evaluation	Reviewed source code from the Draft & Verify paper [16]. Ported BO logic over to LayerSkip to perform DLS w/ BO. Ran evaluations for DLS w/ BO over different <code>exit_layer</code> parameters and compiled resulting metrics.

Table 5: Contributions of team members.

7. Team Contributions

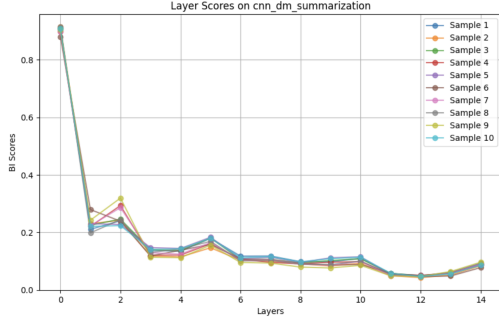
See Table 5.

References

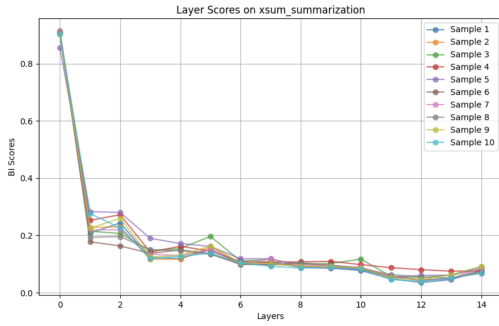
- [1] Josh Achiam, Steven Adler, Sandhini Agarwal, Lama Ahmad, Ilge Akkaya, Florencia Leoni Aleman, Diogo Almeida, Janko Altschmidt, Sam Altman, Shyamal Anadkat, et al. Gpt-4 technical report. *arXiv preprint arXiv:2303.08774*, 2023. 1
- [2] Mostafa Elhoushi, Akshat Shrivastava, Diana Liskovich, Basil Hosmer, Bram Wasti, Liangzhen Lai, Anas Mahmoud, Bilge Acun, Saurabh Agarwal, Ahmed Roman, Ahmed Aly, Beidi Chen, and Carole-Jean Wu. Layerskip: Enabling early exit inference and self-speculative decoding. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, page 12622–12642. Association for Computational Linguistics, 2024. 1, 2, 3, 4, 6, 11
- [3] Angela Fan, Edouard Grave, and Armand Joulin. Reducing transformer depth on demand with structured dropout. *arXiv preprint arXiv:1909.11556*, 2019. 2, 3
- [4] Aaron Grattafiori, Abhimanyu Dubey, Abhinav Jauhri, Abhinav Pandey, Abhishek Kadian, and Ahmad Al-Dahle et al. The llama 3 herd of models, 2024. 1
- [5] Edward J Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. Lora: Low-rank adaptation of large language models. *arXiv preprint arXiv:2106.09685*, 2021. 2
- [6] Gao Huang, Yu Sun, Zhuang Liu, Daniel Sedra, and Kilian Q Weinberger. Deep networks with stochastic depth. In *Computer Vision—ECCV 2016: 14th European Conference, Amsterdam, The Netherlands, October 11–14, 2016, Proceedings, Part IV 14*, pages 646–661. Springer, 2016. 2
- [7] Yann LeCun, John Denker, and Sara Solla. Optimal brain damage. *Advances in neural information processing systems*, 2, 1989. 2
- [8] Yaniv Leviathan, Matan Kalman, and Yossi Matias. Fast inference from transformers via speculative decoding, 2023. 2
- [9] Xinyin Ma, Gongfan Fang, and Xinchao Wang. Llm-pruner: On the structural pruning of large language models. *Advances in neural information processing systems*, 36:21702–21720, 2023. 2, 3
- [10] Xin Men, Mingyu Xu, Qingyu Zhang, Bingning Wang, Hongyu Lin, Yaojie Lu, Xianpei Han, and Weipeng Chen. Shortgpt: Layers in large language models are more redundant than you expect. *arXiv preprint arXiv:2403.03853*, 2024. 2, 3
- [11] Ramesh Nallapati, Bowen Zhou, Caglar Gulcehre, Bing Xiang, et al. Abstractive text summarization using sequence-to-sequence rnns and beyond. *arXiv preprint arXiv:1602.06023*, 2016. 4, 6, 7, 8, 11
- [12] Shashi Narayan, Shay B Cohen, and Mirella Lapata. Don’t give me the details, just the summary! topic-aware convo-

- lutional neural networks for extreme summarization. *arXiv preprint arXiv:1808.08745*, 2018. 4, 6, 8, 11
- [13] Srivastava Nitish. Dropout: a simple way to prevent neural networks from overfitting. *J. Mach. Learn. Res.*, 15:1, 2014. 2
- [14] Fernando Nogueira. Bayesian Optimization: Open source constrained global optimization tool for Python, 2014–. 1, 2
- [15] Qiwei Peng, Yekun Chai, and Xuhong Li. Humaneval-xl: A multilingual code generation benchmark for cross-lingual natural language generalization. *arXiv preprint arXiv:2402.16694*, 2024. 3, 4, 5, 6, 8
- [16] Jun Zhang, Jue Wang, Huan Li, Lidan Shou, Ke Chen, Gang Chen, and Sharad Mehrotra. Draft & verify: Lossless large language model acceleration via self-speculative decoding. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, page 11263–11282. Association for Computational Linguistics, 2024. 2, 5, 9

8. Appendix

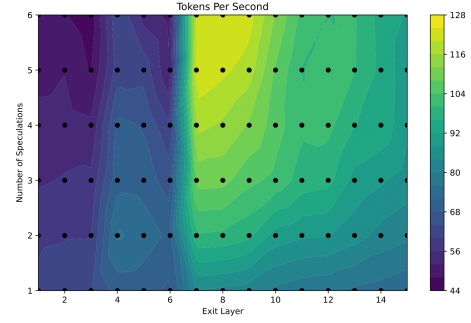


(a) CNN/DM Summarization [11]

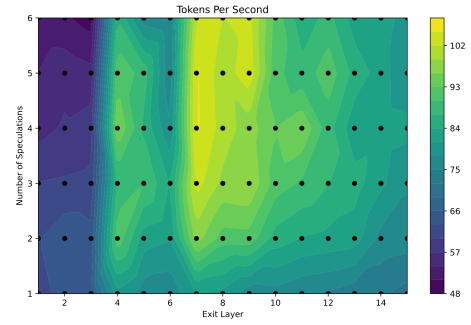


(b) XSUM Summarization [12]

Figure 14: BI Scores on CNN/DM and XSUM across different tasks



(a) CNN/DM Summarization [11]



(b) XSUM Summarization [12]

Figure 16: Layerskip throughput for other datasets.

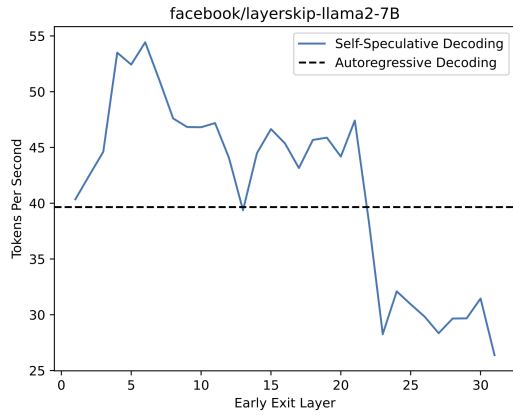


Figure 15: Sweep of the `exit_layer` parameter made by the authors of LayerSkip [2].

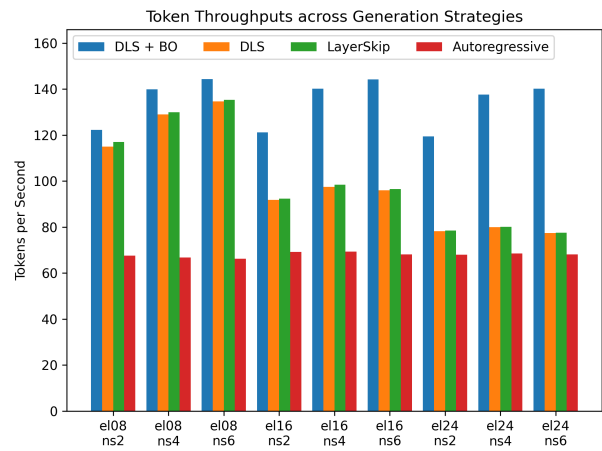


Figure 17: Sweep of a subset of parameters from Figure 7 but with 512 max generation tokens.